



IMU Noise Model

[Jump to bottom](#)

Patrick Geneva edited this page on Mar 28, 2023 · 11 revisions

The [Camera-IMU calibration](#) routine needs to know how "noisy" your IMU is. This is specified in your [IMU configuration YAML file](#) before you start the calibration. Here, you can learn how to set these parameters and how to interpret them.

The IMU Noise Model

The IMU measurement model used in Kalibr contains two types of sensor errors: n , an additive noise term that fluctuates very rapidly ("white noise"), and b , a slowly varying sensor bias. The angular rate measurement $\tilde{\omega}$ (for one single axis of the gyro, in this case) is therefore written as:

$$\tilde{\omega}(t) = \omega(t) + b(t) + n(t)$$

The same model is independently used to model all three sensor axes. The same model (with different parameters, as we will later see) is also used to model the accelerometer measurement errors (on each axis independently). This model is tractable and often used to model inertial sensors [1], [2].

Additive "White Noise"

The rapid fluctuations in the sensor signal are modelled heuristically with a zero-mean, independent, continuous-time white Gaussian noise process $n(t)$ of strength σ_g :

$$E[n(t)] \equiv 0$$

$$E[n(t_1)n(t_2)] = \sigma_g^2 \delta(t_1 - t_2)$$

In other words, the higher σ_g is, the more "noisy" your gyro measurements. The parameters `gyroscope_noise_density` and `accelerometer_noise_density` in the [YAML file](#) specify exactly these noise strengths σ_g (gyro) and σ_a (accel) for the continuous-time model.

This process can be simulated in **discrete-time** as follows:

$$n_d[k] = \sigma_{g_d} w[k]$$

with

$$w[k] \sim N(0, 1)$$

$$\sigma_{g_d} = \sigma_g \frac{1}{\sqrt{\Delta t}}$$

where Δt is the sampling time. This is identical to the discrete-time implementation within Kalibr.

Note: This assumes that the noise was filtered with an ideal low-pass filter that filters noise above $f = \frac{1}{2\Delta t}$ (in other words, an ideal decimation stage). This may or may not be the case, depending on your sensor settings. If you simply "subsample" your gyro or accel, you are not allowed to scale your "white noise density" in that way.

How you can determine this parameter for your particular IMU is explained below.

Bias

In Kalibr, slow variations in the sensor bias are modelled with a "Brownian motion" process, also termed a "Wiener process", or "random walk" in discrete-time. Formally, this process is generated by integrating "white noise" of strength σ_{b_g} (gyro) or σ_{b_a} (accel):

$$\dot{b}_g(t) = \sigma_{b_g} w(t)$$

where w is "white noise" of unit strength.

The parameters `gyroscope_random_walk` and `accelerometer_random_walk` in the [YAML file](#) specify these noise strengths σ_{b_g} and σ_{b_a} . The higher the bias variations in your gyro or accel are, the higher these parameters need to be set.

This process can be simulated in discrete-time as follows:

$$b_d[k] = b_d[k-1] + \sigma_{bgd} w[k]$$

with

$$w[k] \sim N(0, 1)$$

$$\sigma_{bgd} = \sigma_{b_g} \sqrt{\Delta t}$$

This corresponds to the implementation in Kalibr.

The Noise Model Parameters in Kalibr

In Kalibr, the sensor errors are modelled for each sensor axis independently. Unless you are using inertial sensors whose axes have very different noise properties, this is legitimate. Internally, Kalibr will therefore model the "white" noise processes as follows:

$$E[n(t)] = 0_{3 \times 1}$$

$$E[n(t_1)n^T(t_2)] = \sigma_g^2 I_{3 \times 3} \delta(t_1 - t_2)$$

The bias variations ("random walks") are also modelled independently on each sensor axis. Table 1 summarizes all the parameters, and links them to the entries that you can specify in the [YAML file](#). The units of the "sigmas" can also be found in [3].

Table 1: Summary of the IMU noise model parameters as they can be specified in the [IMU configuration YAML file](#) of Kalibr. A discussion of these units can be found [in this thread](#) for those interested.

Parameter	YAML element	Symbol	Units
Gyroscope "white noise"	gyroscope_noise_density	σ_g	$\frac{rad}{s} \frac{1}{\sqrt{Hz}}$
Accelerometer "white noise"	accelerometer_noise_density	σ_a	$\frac{m}{s^2} \frac{1}{\sqrt{Hz}}$
Gyroscope "random walk"	gyroscope_random_walk	σ_{b_g}	$\frac{rad}{s^2} \frac{1}{\sqrt{Hz}}$
Accelerometer "random walk"	accelerometer_random_walk	σ_{b_a}	$\frac{m}{s^3} \frac{1}{\sqrt{Hz}}$
IMU sampling rate	update_rate	$\frac{1}{\Delta t}$	Hz

How to Obtain the Parameters for your IMU

This section describes how you can obtain the Kalibr IMU noise model parameters for your particular IMU. While there are many methods available, we focus on how to get the parameters from the datasheet or using the "Allan standard deviation".

From the Datasheet of the IMU

White Noise Terms: The parameters for the "white noise" processes (σ_g , σ_a) are often specified in the datasheet of the sensor manufacturer. A bit misleading, they are commonly denoted as **angular random walk** in case of the gyro, and **velocity random walk** for the accel. The name comes from the fact that if this white noise on rate or acceleration is integrated (in the navigation equations), it becomes a "random walk" on the angle or the velocity.

Other manufacturers specify it directly as **rate noise density**, **acceleration noise density**, or simply **noise density**. The name comes from the fact that σ_g^2 corresponds to the power spectral *density* of n . Using the discrete-time implementation outlined above, you can check if you interpreted the values in the datasheet correctly.

Bias Terms: In contrast to the "white noise sigmas", σ_{b_g} and σ_{b_a} are rarely directly specified in the datasheet. The reason is that in practice, the bias does not truly behave like a "random walk" for longer integration times. Often, the so-called **in-run bias (in)stability** is specified instead. This sensor parameter indicates (approximately) the accuracy with which the bias can be determined (if a random process is the sum of "white noise" and a "random walk" bias, the bias can not be estimated with arbitrarily low uncertainty at any point in time). In combination with the strength of the "white noise", however, one can often use the in-run bias stability (the lowest point in the Allan standard deviation, see below) to determine reasonable values for σ_{b_g} and σ_{b_a} (assuming that the noise is dominated by "white noise" and a "random walk").

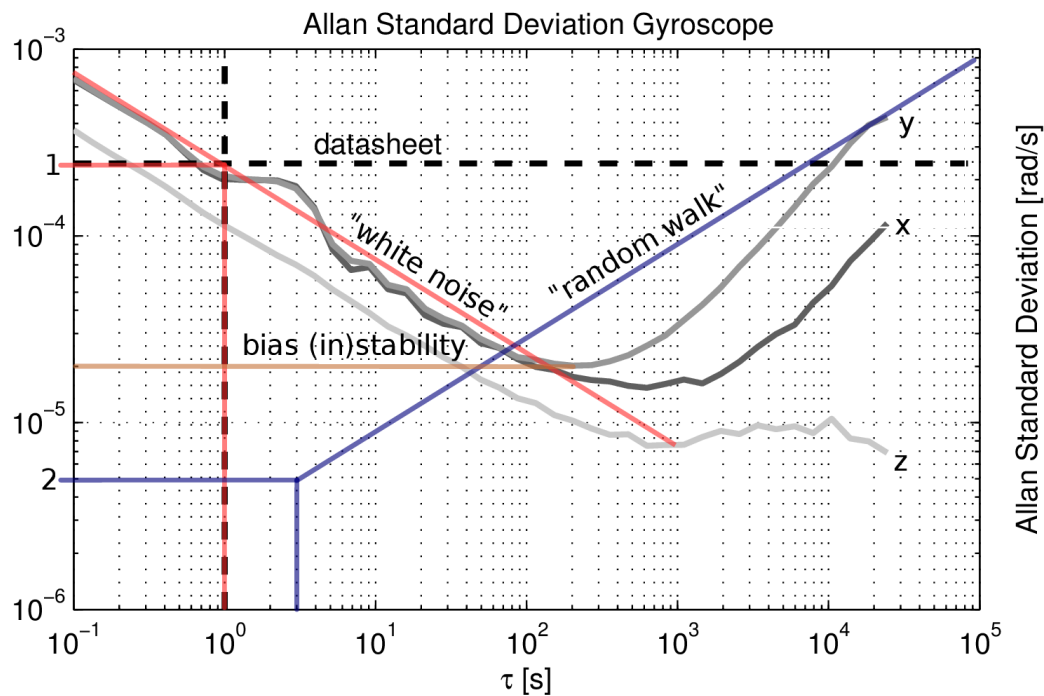
From the Allan standard deviation (AD)

While many (parametric and non-parametric) methods have been proposed to (automatically) determine the noise model parameters from samples, deriving the parameters from an Allan standard deviation plot is probably the most common and standardized procedure. [1] derives the AD for different random processes, including "white noise" (slope -1/2 in a log-log AD plot) and "random walk" (slope +1/2 in a log-log AD plot) that are used in Kalibr. The noise model parameters can be determined directly from the Allan standard deviation.

σ_g and σ_a correspond to the values at $AD(\tau = 1s)$ (point (1) in the figure below). This is only true since the noise power in most inertial sensors is dominated by "white noise" at a frequency of approximately 1Hz.

σ_{b_g} and σ_{b_a} are identified as the value of the (fitted) "random walk" diagonal at an integration time of $\tau = 3s$ (point (2) in the figure below). This can be seen immediately when the Allan standard deviation is derived for a "random walk" process [1].

Figure 1: Allan standard deviation of a MEMS gyro with manually identified noise processes.



Kalibr IMU Noise Parameters in Practice

Some manufacturers provide an Allan standard deviation plot in the datasheet of the device. Otherwise, it needs to be computed from sensor data. A useful open source tool for computing IMU parameters using Allan Deviation is ori-drs/allan_variance_ros which seems to be actively maintained. We recommend using this tool directly on ~15-24 hour dataset recording of the IMU being stationary.

It is important to note that the IMU measurement error model used here is derived from a sensor which does not undergo motion, and at constant temperature. Hence scale factor errors and bias variation caused by temperature changes, for example, are not accounted for. So clearly, the model is optimistic. Particularly when using low-cost MEMS IMUs with Kalibr, you may have to increase the noise model parameters to "capture" these errors as well. In other words, if you use directly the "sigmas" obtained from static sensor data, Kalibr will tend to trust your IMU measurements too much, and its solution will not be optimal.

From our experience, for lowest-cost sensors, increasing the noise model parameters by a factor of 10x or more may be necessary. If you use Kalibr with such a device, please give us feedback, such that we can develop specific guidelines, device-specific parameter suggestions, or more advanced methods to determine these parameters.

[1] "IEEE Standard Specification Format Guide and Test Procedure for Single-Axis Interferometric Fiber Optic Gyros", IEEE Std 952-1997, 1998 <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=660628&isnumber=14457>

[2] "Indirect Kalman Filter for 3D Attitude Estimation", Nikolas Trawny and Stergios I. Roumeliotis, MARS Lab Tech. Report Nr. 2005-002, Rev. 57 <http://mars.cs.umn.edu/tr/reports/Trawny05b.pdf>

[3] "Sigma-Point Kalman Filtering for Integrated GPS and Inertial Navigation", John L. Crassidis, University at Buffalo http://www.acsu.buffalo.edu/~johnc/gpsins_gnc05.pdf

► Pages 18

[Home](#)
[Downloads](#)
[Installation](#)

Toolbox Tools

[Multiple camera calibration](#)
[Camera-IMU calibration](#)
[Multi-IMU and IMU intrinsic calibration](#)
[Rolling Shutter camera calibration](#)

(only ROS):

[Camera focus](#)
[Calibration validator](#)
[ROS2 support](#)

Formats

[Supported camera models](#)
[Calibration targets](#)
[Bag format](#)
[YAML formats](#)
[IMU Noise Model](#)

Tutorials

[Example: Calibrating a VI-Sensor](#)
[Example: Calibrating RealSense Cameras](#)

Experimental

[Camera-IMU-LRF calibration](#)

Clone this wiki locally

<https://github.com/ethz-asl/kalibr.wiki.git>

